PROGETTAZIONE DI SOFTWARE SICURO

Lezione 9

Criteri di copertura
Metodi MCC e MCDC
Altri criteri

Elvinia Riccobene

Università degli Studi di Milano - SSRI

Argomenti

Statement coverage

Branch coverage

Decision coverage

Condition Coverage

Altri criteri di copertura delle condizioni

- MCC
- MCDC

- **Testing con classi di equivalenza**
- **State-based Testing**
- **Path Testing**

Criteri di test strutturali

Ricorda che criterio: P x T → {true,false} può essere anche inteso come generatore di test set dato P : C: P → T (tale che C(P,T) = true)

Nel program-based testing, il criterio di selezione dei test si deriva dalla struttura stessa del programma (in particolare, dal grafo di flusso)

Vedremo i seguenti criteri di test basati sulla copertura dei programmi:

1. **copertura delle istruzioni (statement coverage)**
2. **copertura degli archi (branch coverage)**

Statement coverage

Un test set T e' adeguato per testare un programma P secondo il criterio di copertura delle istruzioni, se per ogni istruzione s di P esiste un caso di test in T che esegue s

Ogni istruzione viene eseguita almeno una volta

Esempio:

- `if x /= 0 then x:=x+10 else x:=x-10`

Perchè T possa soddisfare la copertura delle istruzioni deve contenere almeno due casi, uno con x = 0 e uno con x /= 0, ad esempio

T = { x = 0, x = 10}

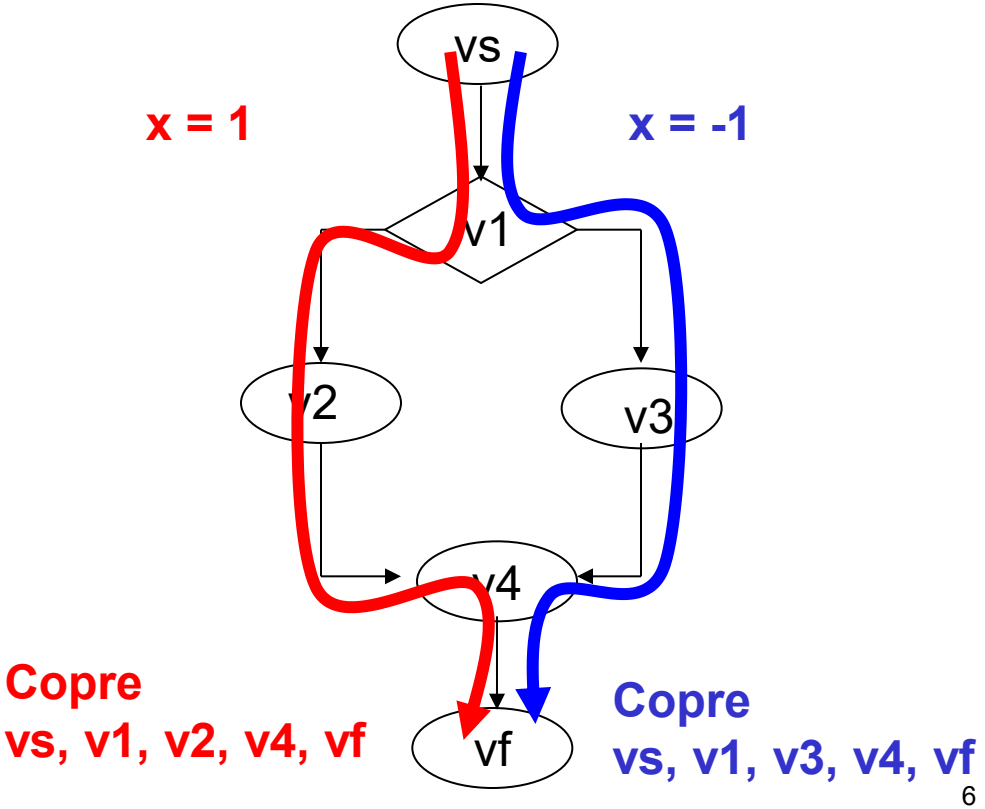
Statement Coverage – Esempio (1)

Istruzioni

- vs, v1, v2, v3, v4, vf

```
vs: int main(int x) {
     int result;

v1:  if (x > 0)
v2:      result = x;
     else
v3:      result = 1 / x;
v4:  printf("%d", result);
vf: }
```

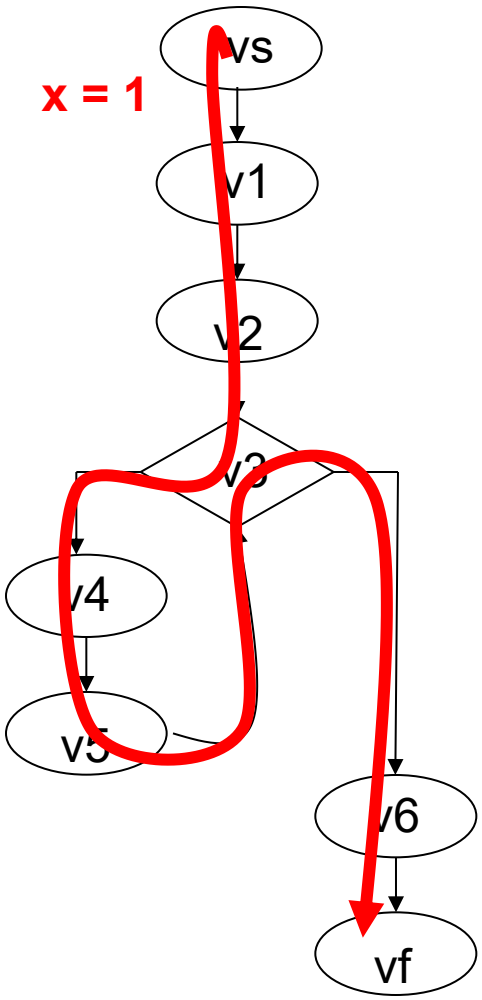


Statement coverage – Esempio (2)

Istruzioni

- vs, v1, v2, v3, v4, v5, v6, vf

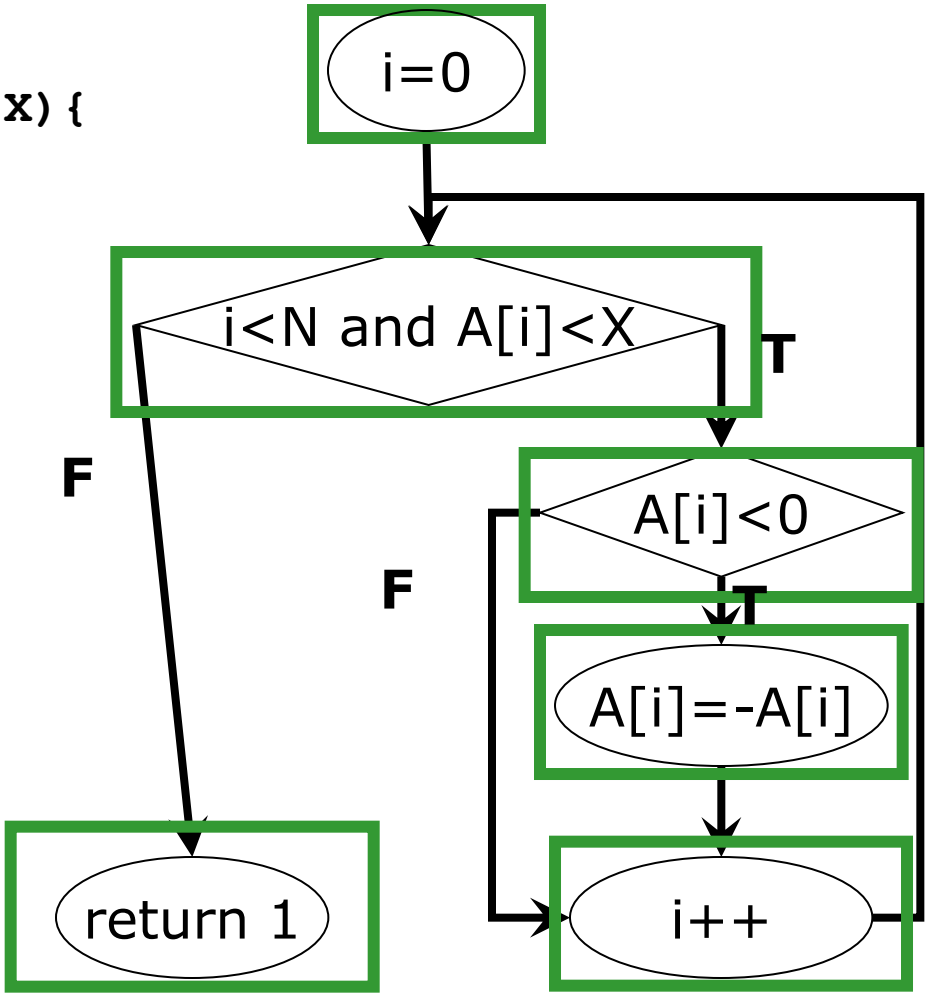
```
vs:  int main(int x) {
       int sum, I;
v1:    sum = 0;
v2:    i = 1;
v3:    while (i <= x) {
v4:        sum = sum + i;
v5:        i = i + 1;
       }
v6:    printf("%d",sum);
vf:  }
```



Statement coverage – Esempio (3)

```
int select(int A[], int N, int X){
  int i=0;
  while (i<N and A[i] <X) {
     if (A[i]<0)
          A[i] = - A[i];
     i++;
  }
  return(1);
}
```

Il caso di test:
(N=1, A[0]=-7, X=9)
è sufficiente



Fault detection capability

Come valutare un criterio di copertura?

- Fault detection capability: quali errori è in grado di trovare e (più importante) quali garantisce di trovare un set test adeguato al criterio

Statement coverage:

istruzioni (sempre) errate vengono individuate

- gli errori nelle decisioni non è detto che vengano trovati
- questo criterio è debole
- però potrebbe anche bastare: idea di "MINIMAL CRITERION FOR COMMERCIAL SOFTWARE"

Misura di copertura dei comandi

- Un caso di test (o una test suite) può soddisfare parzialmente un criterio di copertura
- Va quindi definita una **misura di copertura:** P x T -> [0, 1]
- Per il criterio di copertura delle istruzioni la **misura di copertura** è definita:

$$C_0 = \frac{\text{n. di statement eseguiti}}{\text{n. di st. eseguibili}}$$

Una test suite *T* **soddisfa** (o è *adeguato* per) il criterio di copertura delle istruzioni

se $C_0 = 1$

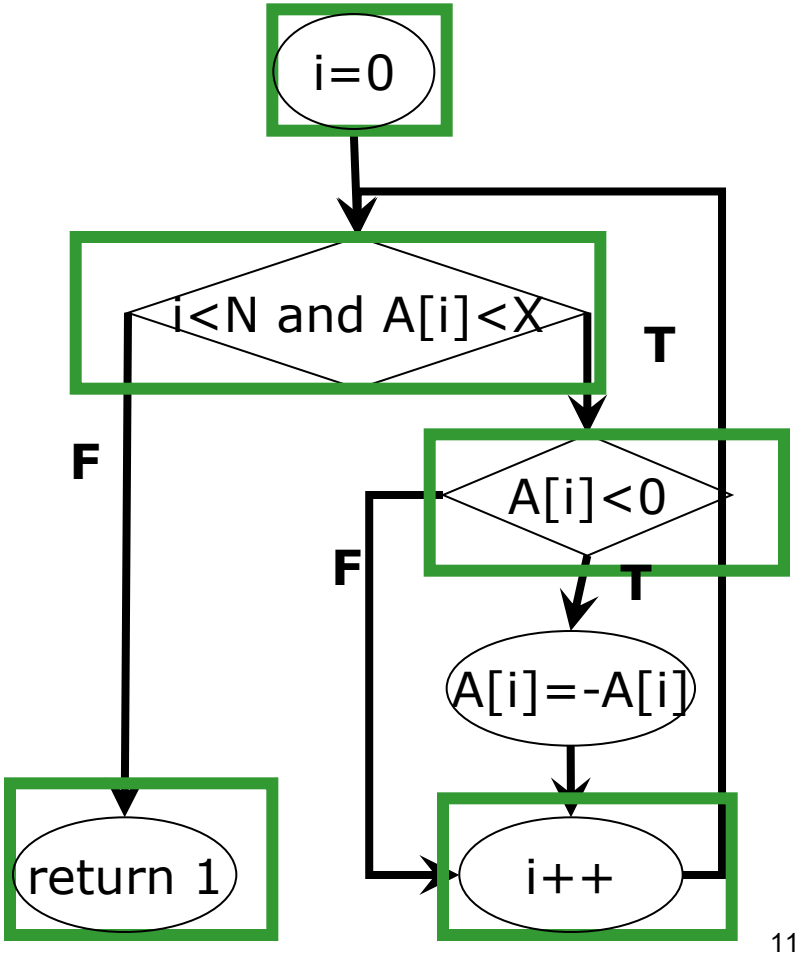
Misura di copertura dei comandi

Il caso di test: (N=1, A[0]=7, X=9)

copre 5/6, quindi è parzialmente adeguato al criterio

Il caso di test: (N=1, A[0]=-7, X=9)

copre 6/6, quindi è adeguato al criterio



Branch coverage (o test dei rami)

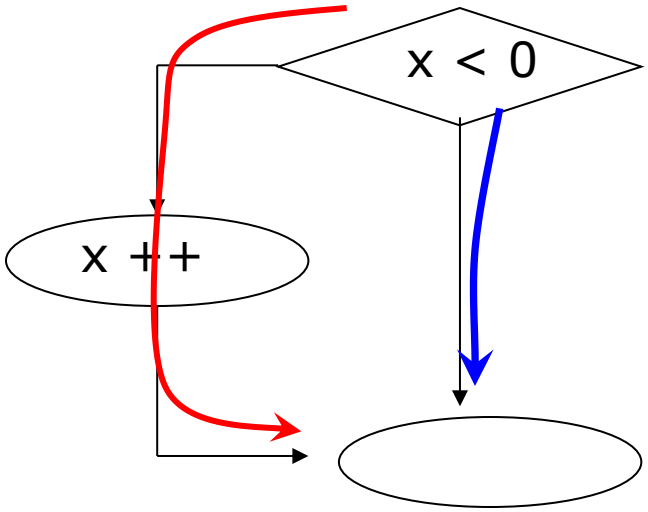
Un test set T soddisfa il criterio di copertura degli archi (o branch coverage) di P se e solo se ogni arco (branch) del grafo di P è percorso almeno una volta

Esempio:

```
if x < 0 then x ++ endif
```

Per ottenere la copertura dei branch ho bisogno di almeno due casi, uno con x < 0 e uno con x >= 0, ad esempio

T = { x = -3, x = 10}



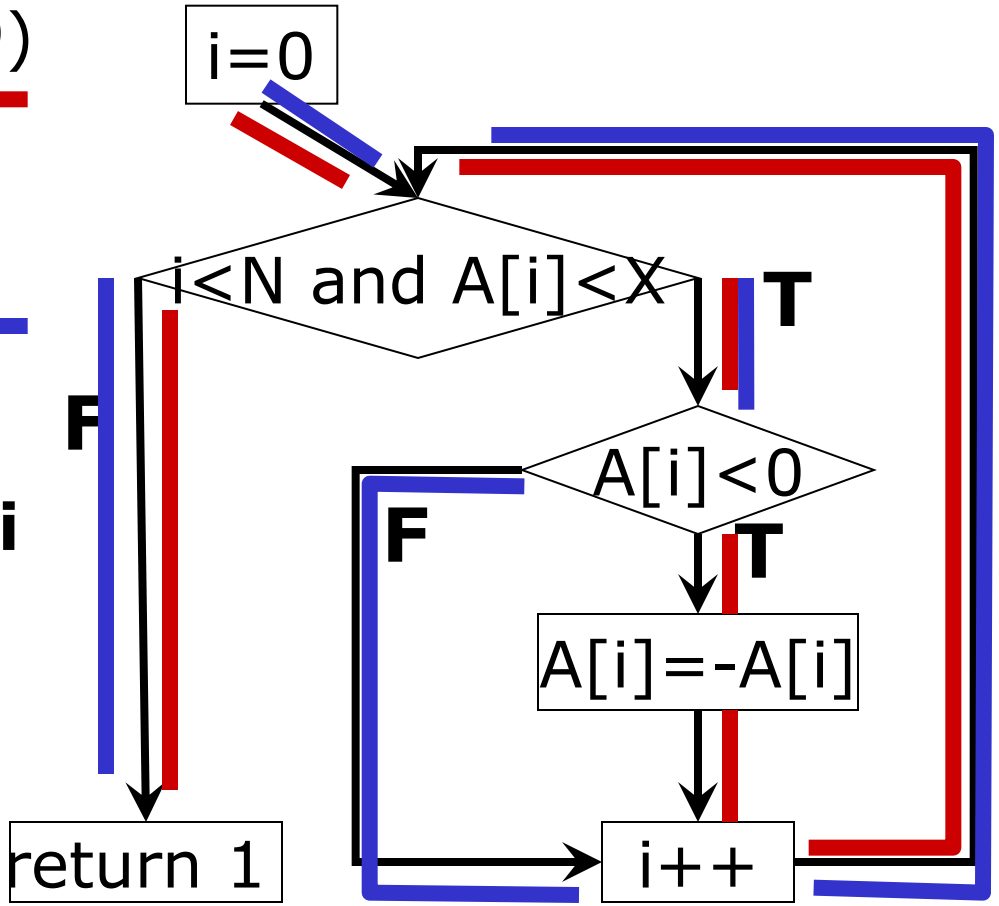
Branch coverage - Esempio

Abbiamo 2 casi di test:

- (N=1, A[0]=-7, X=9)
- (N=1, A[0]=7, X=9)

Se prendiamo il caso di test:

(N=2, A=[-7,7], X=9)**?**

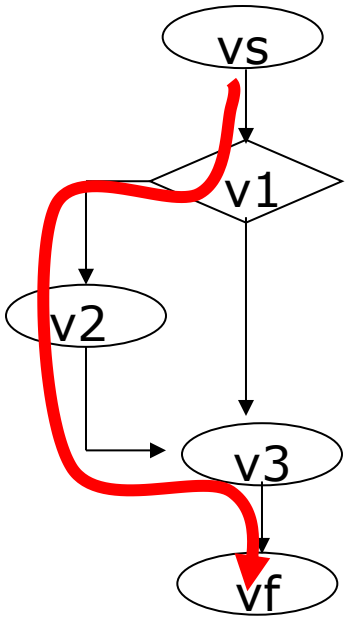


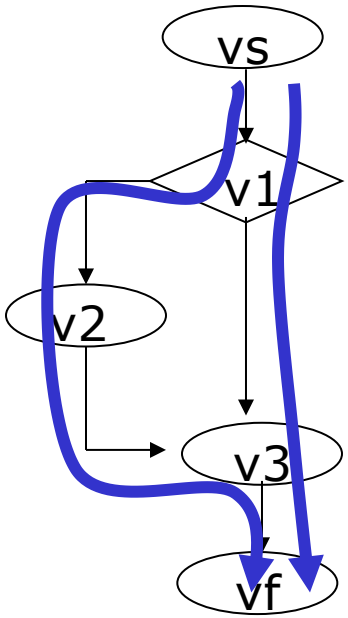
Differenza tra Statement e Branch coverage

Statement coverage è più debole

Il branch coverage implica lo statement cov.

- se un test set T soddisfa il branch coverage, soddisfa anche il statement coverage
- ma non viceversa





Statement coverage **Branch coverage**

Misura di copertura degli archi

Un test T può soddisfare il criterio degli archi parzialmente. In quel caso si misura la copertura:

$$C_{path} = \frac{\text{numero di branch eseguiti}}{\text{numero di branch eseguibili}}$$

Una test suit soddisfa il criterio di adeguatezza del branch coverage se $C_{path} = 1$

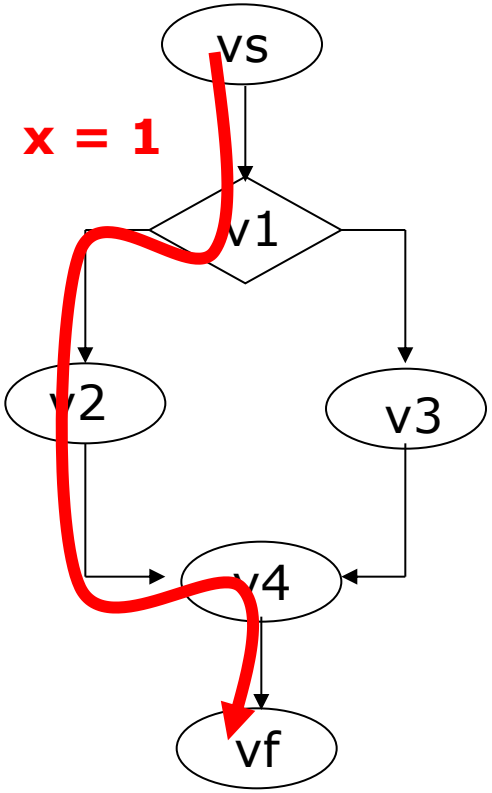
Misura di copertura degli archi

- La test suit T ={1} copre un arco su i due eseguibili, ottenendo la copertura del 50% (quindi è parzialmente adeguato)

- La test suit T ={1, -1} soddisfa il criterio di adeguatezza del branch coverage

```
vs: int main(int x) {
     int result;

v1:  if (x > 0)
v2:      result = x;
     else
v3:      result = 1 / x;
v4:  printf("%d", result);
vf: }
```



Criteri di test strutturali

Abbiamo visto i criteri di copertura:

1. delle istruzioni

- tutte le istruzioni vengono eseguite

2. dei branch

- tutti gli archi nel grafo di flusso vengono percorsi

Vedremo ora

3. delle decisioni

4. delle condizioni

Decisioni e Condizioni (1)

Decisioni

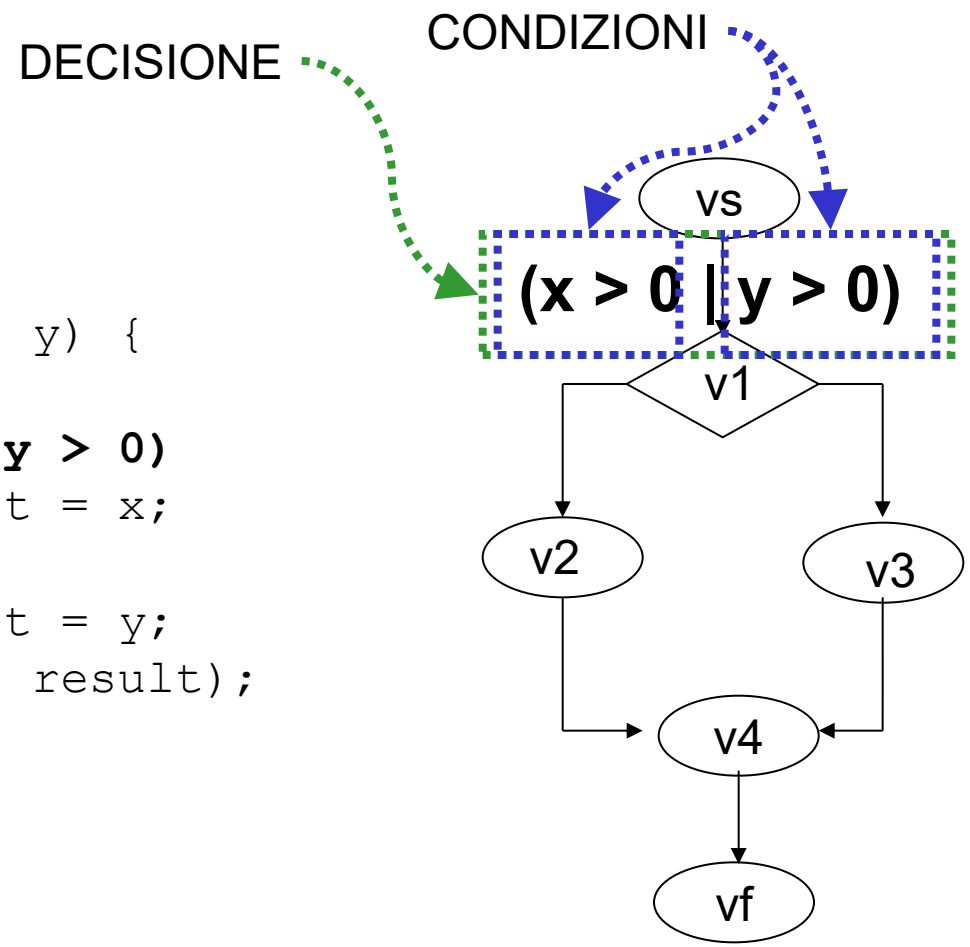
- predicato (espressione booleana) guardia di una istruzione condizionale (if) o di una iterativa (while, for, …)
- **esempio** `if (x > 0 | y > 0)` **…**

Condizioni

- una espressione booleana atomica (cioè non divisibile in altre espressioni più semplici) che appare in una decisione
- **nell'esempio sopra** `x > 0` e `y > 0`

Decisioni e Condizioni (2)

```
vs: int main(int x, y) {
        int result;
v1:     if (x > 0 | y > 0)
v2:             result = x;
        else
v3:             result = y;
v4:     printf("%d", result);
vf:  }
```



Copertura delle decisioni

Un test set T e' adeguato per testare un programma P secondo il criterio di copertura delle decisioni, se per ogni decisione di P esiste

- un caso di test in T in cui la decisione è presa
- un caso di test in T in cui la decisione non è presa

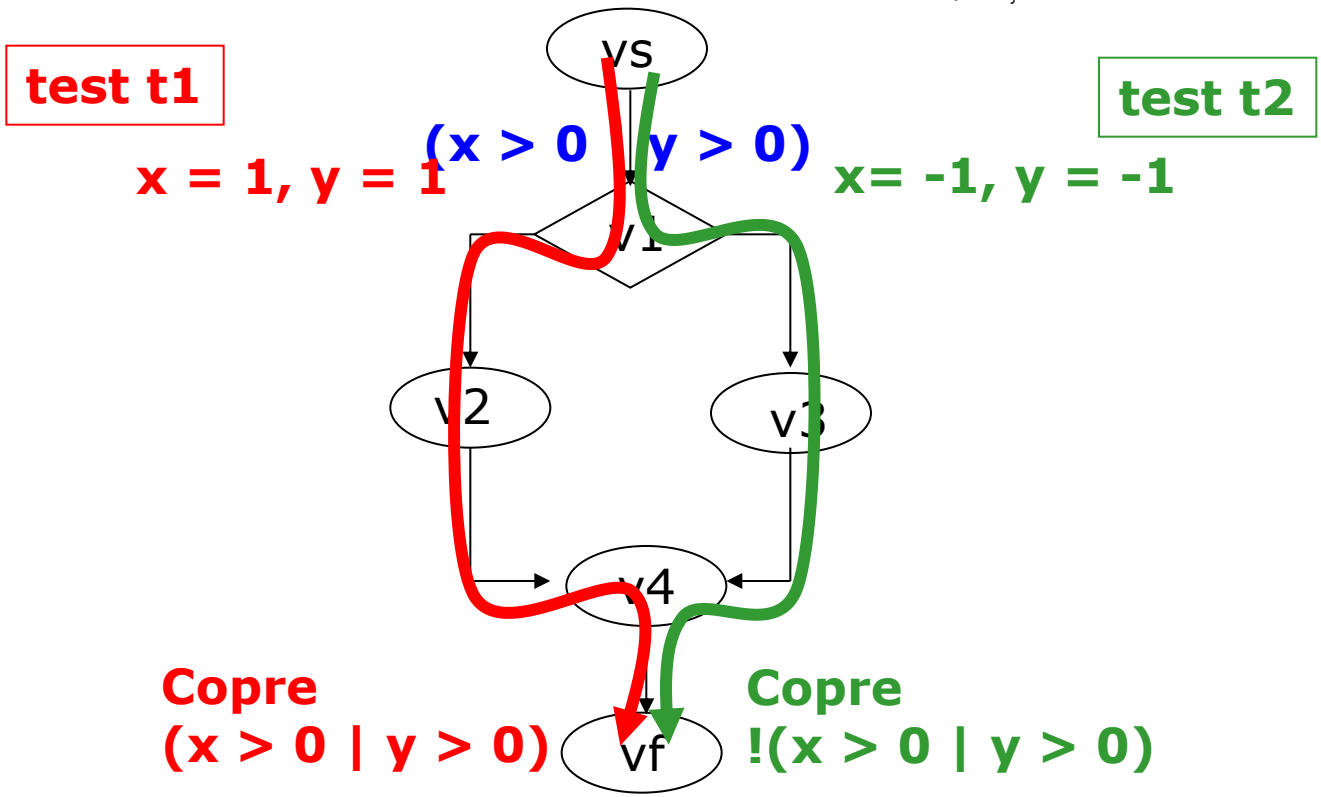
Copre ogni decisione e la sua negazione

E' equivalente al branch coverage

- ogni arco nel grafo di flusso viene percorso

Esempio Copertura delle decisioni

```
vs: int main(int x, y) {
        int result;
v1:     if (x > 0 | y > 0)
v2:             result = x;
        else
v3:             result = y;
v4:     printf("%d", result);
vf:  }
```



Copertura delle Condizioni

Un test set T e' adeguato per testare un programma P secondo il criterio di copertura delle condizioni, se per ogni condizione di P esiste

- un caso di test in T in cui la condizione è vera
- un caso di test in T in cui la condizione è falsa

Copre ogni condizione e la sua negazione

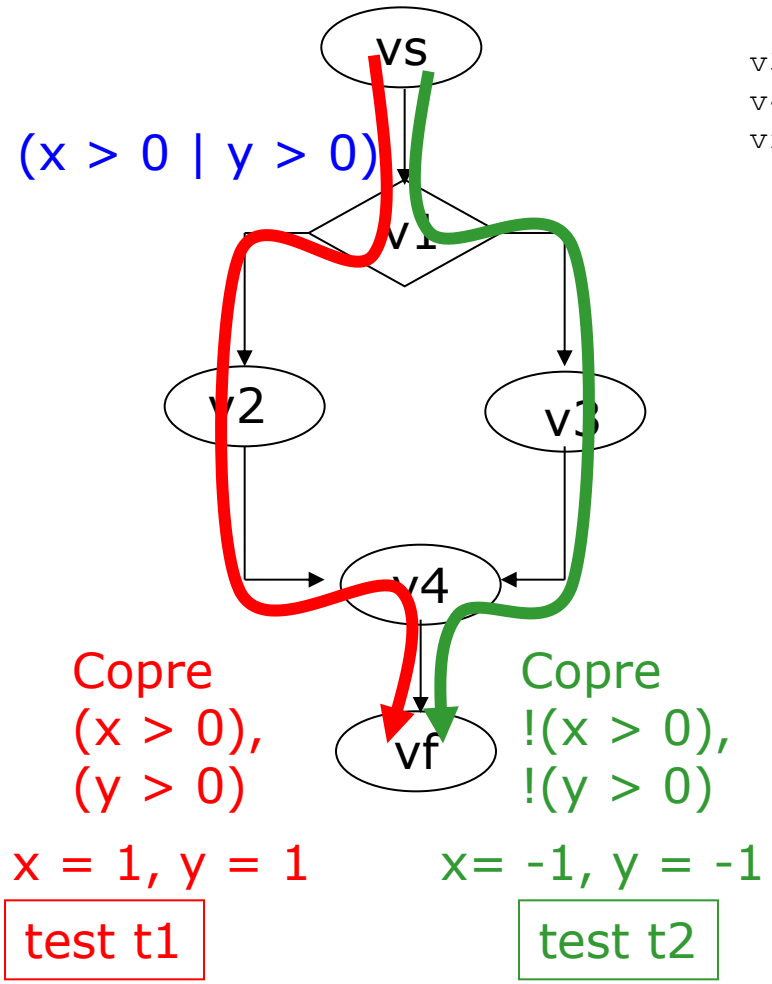
$$C_{cond} = \frac{\text{num di valori di verità assunti da tutte le condizioni}}{2 \text{ X numero di condizioni}}$$

Test suit *T* soddisfa il criterio se $C_{cond} = 1$

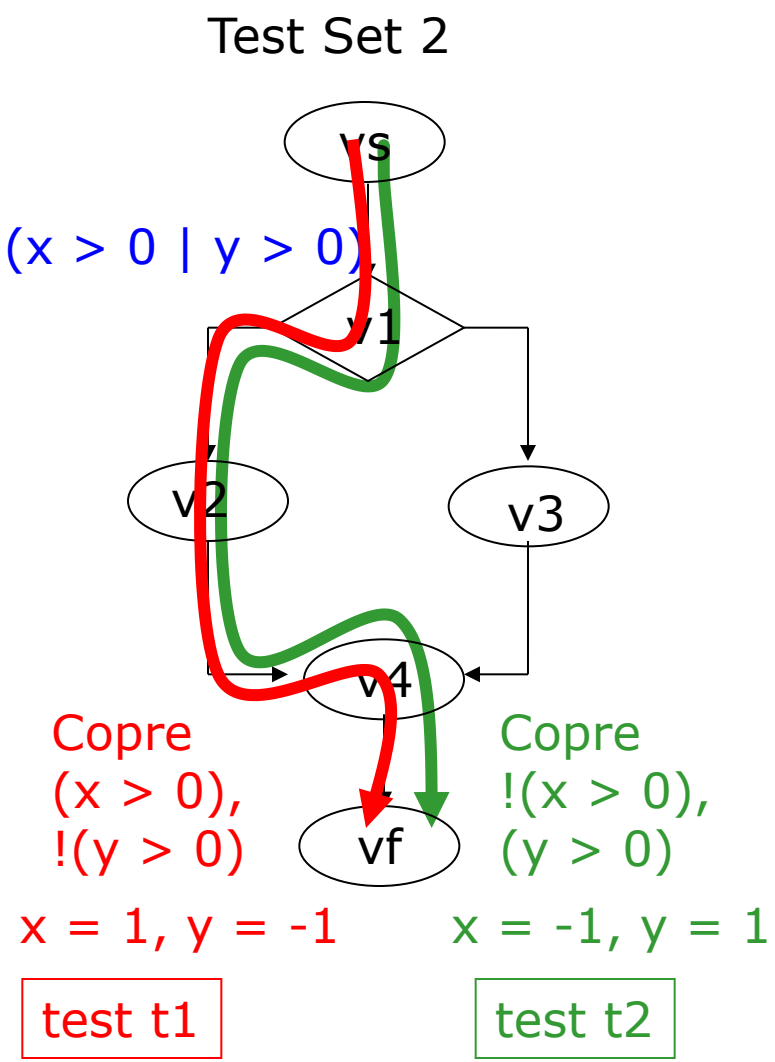
Esempio Copertura delle Condizioni (1)

```
vs: int main(int x, y) {
        int result;
v1:     if (x > 0 | y > 0)
v2:             result = x;
        else
v3:             result = y;
v4:     printf("%d", result);
vf:  }
```

Test Set 1



Esempio Copertura delle Condizioni (2)



Anche Test Set 2 soddisfa la copertura delle condizioni

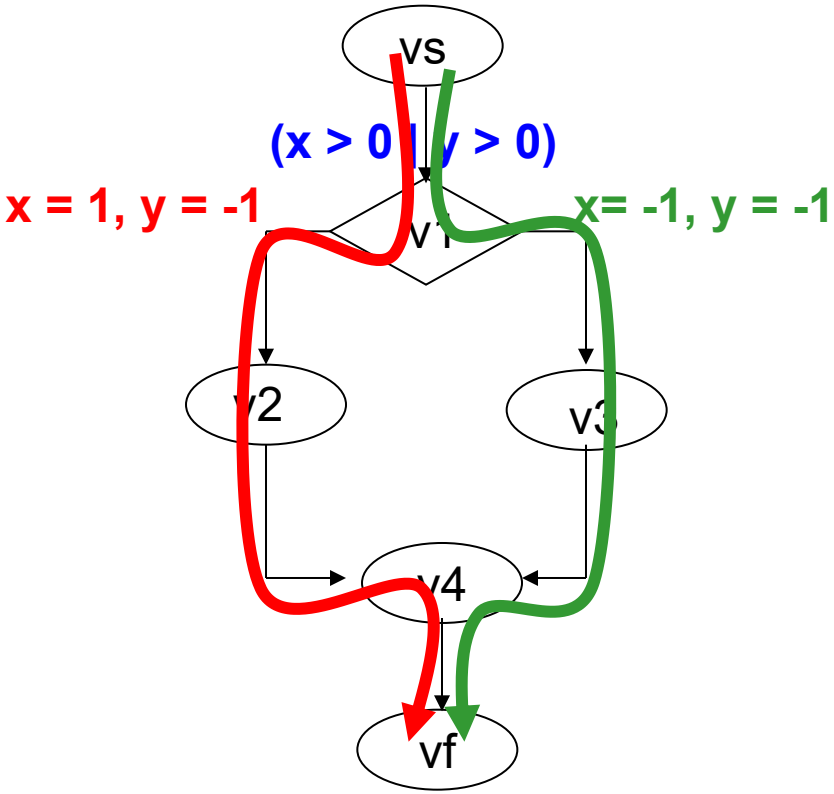
Tuttavia non copre la decisione !!

A. La copertura delle condizioni non implica la copertura delle decisioni

e neache il branch coverage

Esempio Copertura delle Decisioni (2)

Test Set 3



Test set 3 copre le decisioni ma non le condizioni

- y non è mai > 0

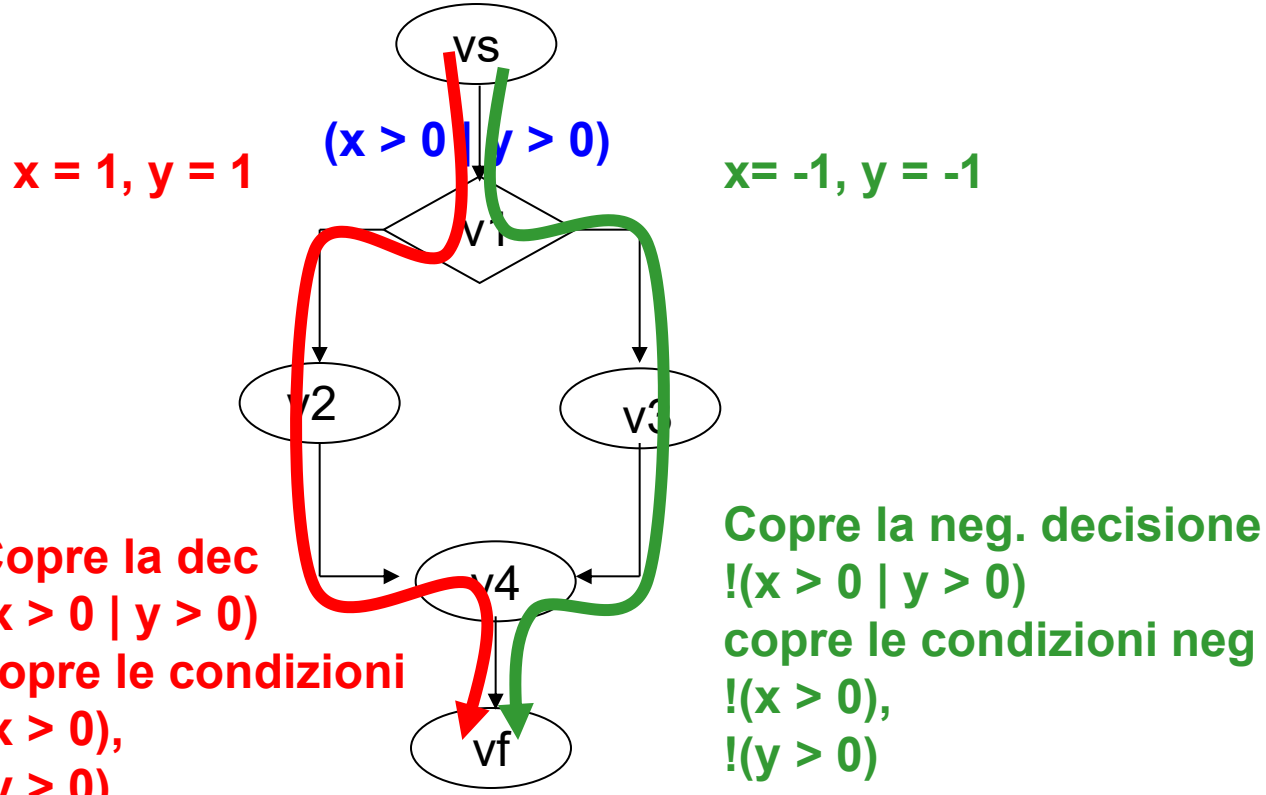
B. La copertura delle decisioni non implica la copertura delle condizioni

QUINDI A + B →

il criterio di copertura delle condizioni e quello delle decisioni sono incomparabili

Copertura Decisioni e Condizioni

Il criterio di copertura delle decisioni e delle condizioni richiede la copertura sia di tutte le **decisioni** che di tutte le **condizioni**



Short circuit evaluation

I compilatori spesso usano per efficienza la valutazione a corto circuito per le espressioni booleane:

Esempi:

- a && b: Se a è falso non valuta b
- a || b: Se a è vero non valuta b

Nota che spesso esistono operatori che evitano ciò (esempio & e | al posto di && e ||)

Questo può complicare la scelta dei casi di test

Esempio con short circuit

Testo ogni decisione/condizione per

`( y > 0 or  x > 0)`

test	y>0	x>0	decisione
y = 1, x = 1	**T**	**non valutata**	**T**
y = 0, x = 1	**F**	**T**	**T**
y = 0, x = 0	**F**	**F**	**F**

- per testare x>0 la prima condizione deve essere falsa, perchè se è fosse vera la condizione x>0 non verrebbe valutata

Criteri di test strutturali

Abbiamo visto i criteri di copertura:

1. delle istruzioni
2. dei branch
3. delle decisioni
4. delle condizioni
5. delle decisioni/condizioni

Vediamo ora come coprire ulteriormente le condizioni all'interno delle decisioni:

6. Multiple condition coverage (MCC)
7. Modified Condition/Decision coverage MCDC

Multiple condition coverage

Un test set soddisfa MCC se testa ogni possibile combinazione dei valori di verità delle condizioni atomiche in ogni decisione

- Con n condizioni in una decisione si avranno 2^n combinazioni da soddisfare (T e F per ogni condizione)
- Si rappresenta in genere con una tabella
- Il numero si riduce grazie alla short circuit evaluation

Esempio Multiple condition cov.

(digit_high == -1 || digit_low == -1)

Test Case	digit_high == -1	digit_low == -1	
1	False	False	00
2	False	True	01
3	True	(non valutato)	10 11

il numero di test cresce esponenzialmente con il numero n di condizioni. Se n è grande l'MCC non è fattibile

Modified Condition/decision coverage

Per questo motivo si sono introdotti altri criteri più forti del semplice condition ma lineari con n.

il Modified Condition/decision coverage MCDC ha avuto successo - standard

Richiesto dalla FAA sul software dei aeromobili commerciali

Biliografia: Chilenski, J., Miller, S. *Applicability of modified condition/decision coverage to software testing.* Software Engineering Journal, September 1994, pp. 193--200.

MCDC

Il test set deve essere preso in modo che ogni condizione all'interno di una decisione deve far variare in modo indipendente il valore finale della decisione

- Il test deve mostrare che ogni condizione atomica influenzi l'esito della decisione in modo indipendente dalla valutazione delle altre condizioni
  - per ogni condizione atomica C, ci sono due casi di test in cui i valori di verità di tutte le condizioni eccetto C sono gli stessi, e la decisione è valutata True per uno di questi casi di test e False per l'altro

Esempio (1)

per ogni condizione C in una decisione D ho due casi di test in cui C varia, le altre condizioni no e il valore finale della decisione D varia

Dec = A and B

- considera A, varia solo A in modo che Dec vari (tengo B vero)
  - [A vero, B vero] (Dec) [A falso, B vero] (not Dec)
- varia B:
  - [A vero, B vero] (Dec) [A vero, B falso] (not Dec)
- dei 4 casi mi bastano 3 {[A,B],[not A,B][A,not B]}

Dec = A or B

- A: [A vero, B falso] (Dec) [A falso, B falso] (not Dec)
- B: [A falso, B vero] (Dec) [A falso, B falso] (not Dec)
- Tre casi {[not A, not B],[A, not B][not A,B]}

Esempio (2)

```
If Reset = on and
   (Pressure = TooLow or Pressure = Normal) then ...
```

Esempio (2)

```
If Reset = on and
   (Pressure = TooLow or Pressure = Normal) then ...
```

Reset = on	Pressure = TooLow	Pressure = Normal	Decisione
T	T	-	T
F	T	- F	F
T	T	F	T
T	F	F	F
T	F	T	T
T	F	F	F

4 casi di test

Test set con MCDC

$$(\overset{A}{\overline{a>2}} \,||\, (\overset{B}{\overline{a>b}} \,\&\, \overset{C}{\overline{a>c}})$$

	A	B	C	D
1	T	F	-(T)	T
~~2=4~~	~~F~~	~~F~~	~~-(T)~~	~~F~~
3	F	T	T	T
4	F	F	T	F
~~5=3~~	~~F~~	~~T~~	~~T~~	~~T~~
6	F	T	F	F

test set che soddisfa MCDC

$$T = \{\overset{1}{(3,4,2)}, \overset{3}{(1,0,0)}, \overset{4}{(1,2,0)}, \overset{6}{(1,0,2)}\}$$

Valutare con MCC

Esercizio

FARE L'ANALISI DELLE DECISIONI E DELLE CONDIZIONI CON L'MCDC PER IL SEGUENTE FRAMMENTO DI PROGRAMMA:

```
foo(int x, int y, int z, int w){
    if((x>5 && y>0) || (z>5))
        then if (x>5 && z>5) then statement1;
        else statement2;
    else statement3;
```

Copertura delle decisioni

$D_1 = (x>5 \;\&\&\; y>0) \;||\; (z>5)$

$T_1 = \{(6,1,4,-),(4,1,4,-)\}$

RENDE TRUE D_1 RENDE FALSE D_1

$D_2 = (x>5 \;\&\&\; z>5)$

$T_2 = \{(6,-,6,-),(4,-,6,-)\}$

RENDE TRUE D_2

RENDE FALSE D_2

NON POSSO RIDURRE $z=6$ SE NO NON PASSEREBBE L'IF ESTERNO

Copertura delle condizioni (1/2)

	x>5	y>0	z>5	D_1	
1	T	T	F	T	
2	F	T	F	F	
~~3~~	~~T~~	~~T~~	~~F~~	~~T~~	3=1
4	T	F	F	F	
5	F	-	T	T	
~~6~~	~~F~~	~~-~~	~~F~~	~~F~~	6=2

$$T_1 = \{(6,\overset{1}{1},4,-),(4,\overset{2}{1},4,-),(6,\underset{4}{-1},4,-),(4,\underset{5}{-},6,-)\}$$

Copertura delle condizioni (2/2)

	x>5	z>5	D_2
1	T	T	T
2	F	T	F
3	~~T~~	~~T~~	~~T~~
4	T	F	F

3 = 1

DEVE GARANTIRE
(x>5 && y>0) || (z>5)

$T_2 = \{(6, -, 6, -), (4, -, 6, -), (6, 1, 4, -)\}$

1 2 4

- tc1 e tc2 coprono la cond x>5 in D_1
- tc1 e tc3 coprono y>0 in D_1

$T = T_1 \cup T_2 = \{\overset{tc1}{(6, 1, 4, -)}, \overset{tc2}{(4, 1, 4, -)}, \overset{tc3}{(6, -1, 4, -)}, \underset{tc4}{(4, -, 6, -)}, \underset{tc5}{(6, -, 6, -)}\}$

tc5 e tc1 coprono z>5 in D_2
tc2 e tc4 coprono z>5 in D_1

Path testing (o test dei cammini)

È una strategia di testing il cui obbiettivo è eseguire ogni percorso indipendente di un programma:

- tutte le istruzioni vengono eseguite almeno una volta;
- tutte le decisioni vengono testate a true e false.

I casi di test vanno presi in modo tale che *ciascun percorso del grafo di flusso sia attraversato almeno una volta*

nb: in branch coverage un test case copre solo un arco

Path testing ⇒ copre sempre il branch testing
Branch testing ⊄ Path testing

Copertura di path testing

Numero minimo di test necessari per la copertura di tutti i path = numero di path indipendenti del grafo di flusso.

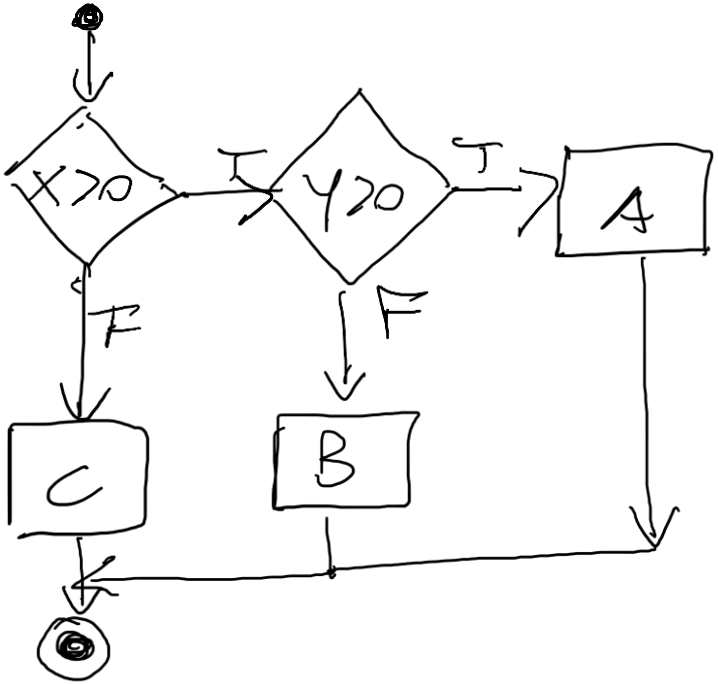
Questo è definito come:

cyclomatic complexity (CC) del grafo di flusso

CC = numero di *archi* - numero di *nodi* + 2

- numero di nodi = numero di decisioni e azioni (+ inizio e fine)
- numero di archi = numero di transizioni nel grafo

Path testing:esempio




```
f(x, y)=
 if x > 0 then
   if y > 0 then print("A")
   else print("B")
 else print("C")
```

TP={(1,1), (1,-1), (-1,_)} copre il path testing con n=7, a=8 e CC=3

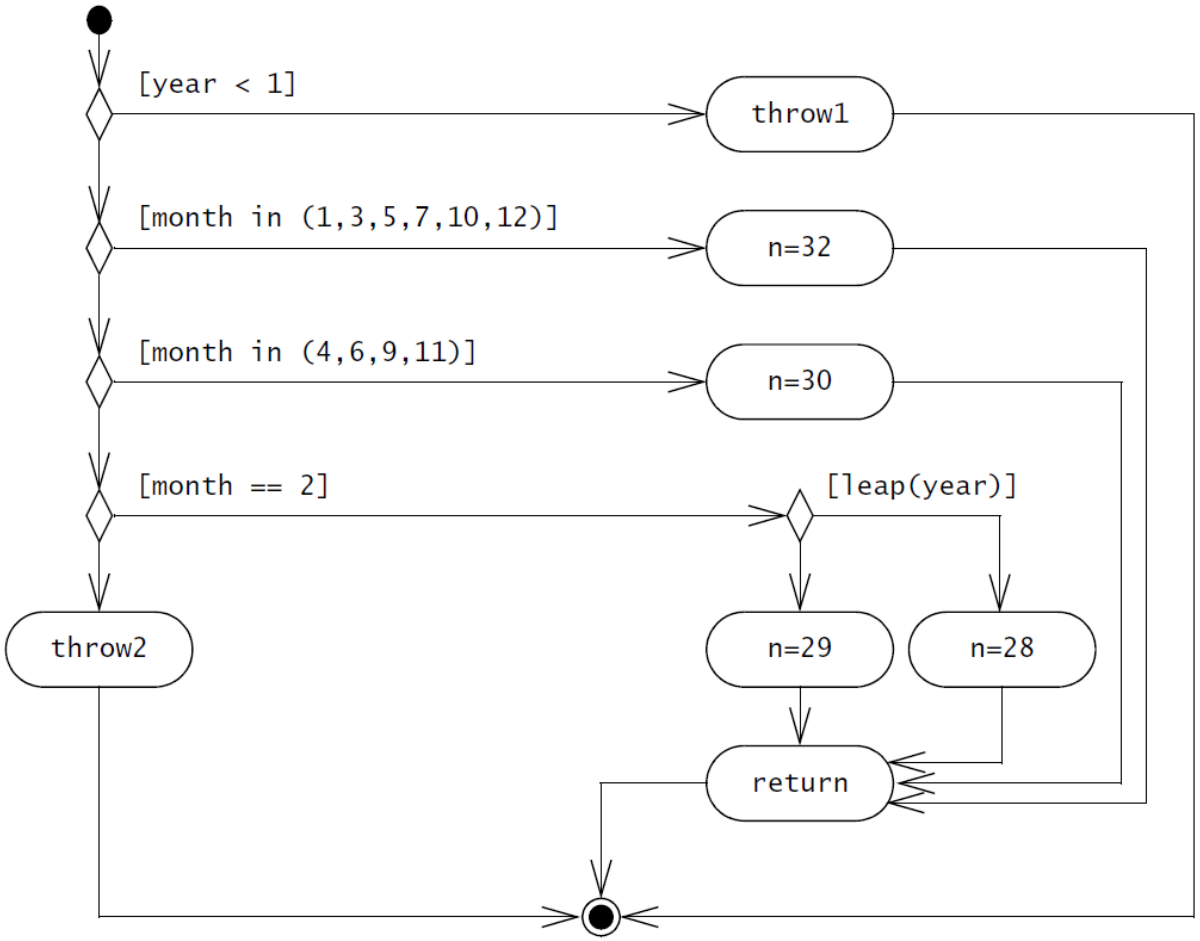
Per il branch occorre coprire (x>0) e (y>0) quindi in principio occorrono 2 casi di test per decisione (anche se i 3 casi di test di TP bastano)

Esempio di codice con fault

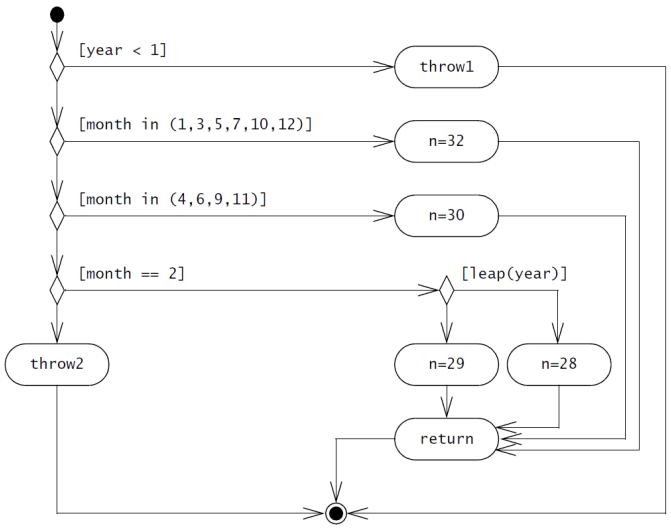
```
public class MonthOutOfBounds extends Exception {…};
public class YearOutOfBounds extends Exception {…};

class MyGregorianCalendar {
    public static boolean isLeapYear(int year) {
        boolean leap;
        if ((year%4) == 0){
            leap = true;
        } else {
            leap = false;
        }
        return leap;
    }
    public static int getNumDaysInMonth(int month, int year)
            throws MonthOutOfBounds, YearOutOfBounds {
        int numDays;
        if (year < 1) {
            throw new YearOutOfBounds(year);
        }
        if (month == 1 || month == 3 || month == 5 || month == 7 ||
                month == 10 || month == 12) {
            numDays = 32;
        } else if (month == 4 || month == 6 || month == 9 || month == 11) {
            numDays = 30;
        } else if (month == 2) {
            if (isLeapYear(year)) {
                numDays = 29;
            } else {
                numDays = 28;
            }
        } else {
            throw new MonthOutOfBounds(month);
        }
        return numDays;
    }
}
```

Grafo di flusso del metodo `getNumDaysInMonth()`



Test suite per path coverage



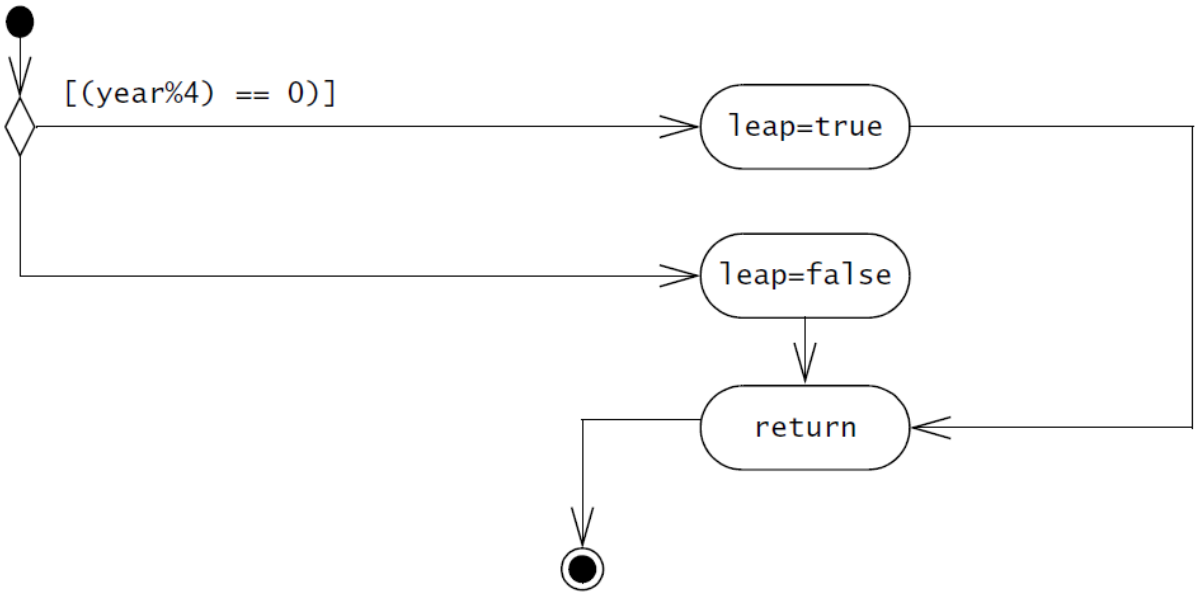
CC del metodo `getNumDaysInMonth()` = 6 che è anche in numero di casi di test trovati

Test case	Path
(year = 0, month = 1)	{throw1}
(year = 1901, month = 1)	{n=32 return}
(year = 1901, month = 2)	{n=28 return}
(year = 1904, month = 2)	{n=29 return}
(year = 1901, month = 4)	{n=30 return}
(year = 1901, month = 0)	{throw2}

Esempio: metodo `isLeapYear`

```
public static boolean isLeapYear(int year) {
    boolean leap;
    if ((year%4) == 0){
        leap = true;
    } else {
        leap = false;
    }
    return leap;
}
```

CC del metodo `isLeapYear()` = 2



Test case	Path
year = 1901, month = 2	*leap=false return*
year = 1904, month=2	leap=true return

Esercizio

```
esempio(a, b)=

  if a > 0  then
     if b > 0 then print("A")
  if b > 0 then print("B")
```

È una funzione per cui un test set adaguato per la copertura dei rami non riesce a coprire i cammini

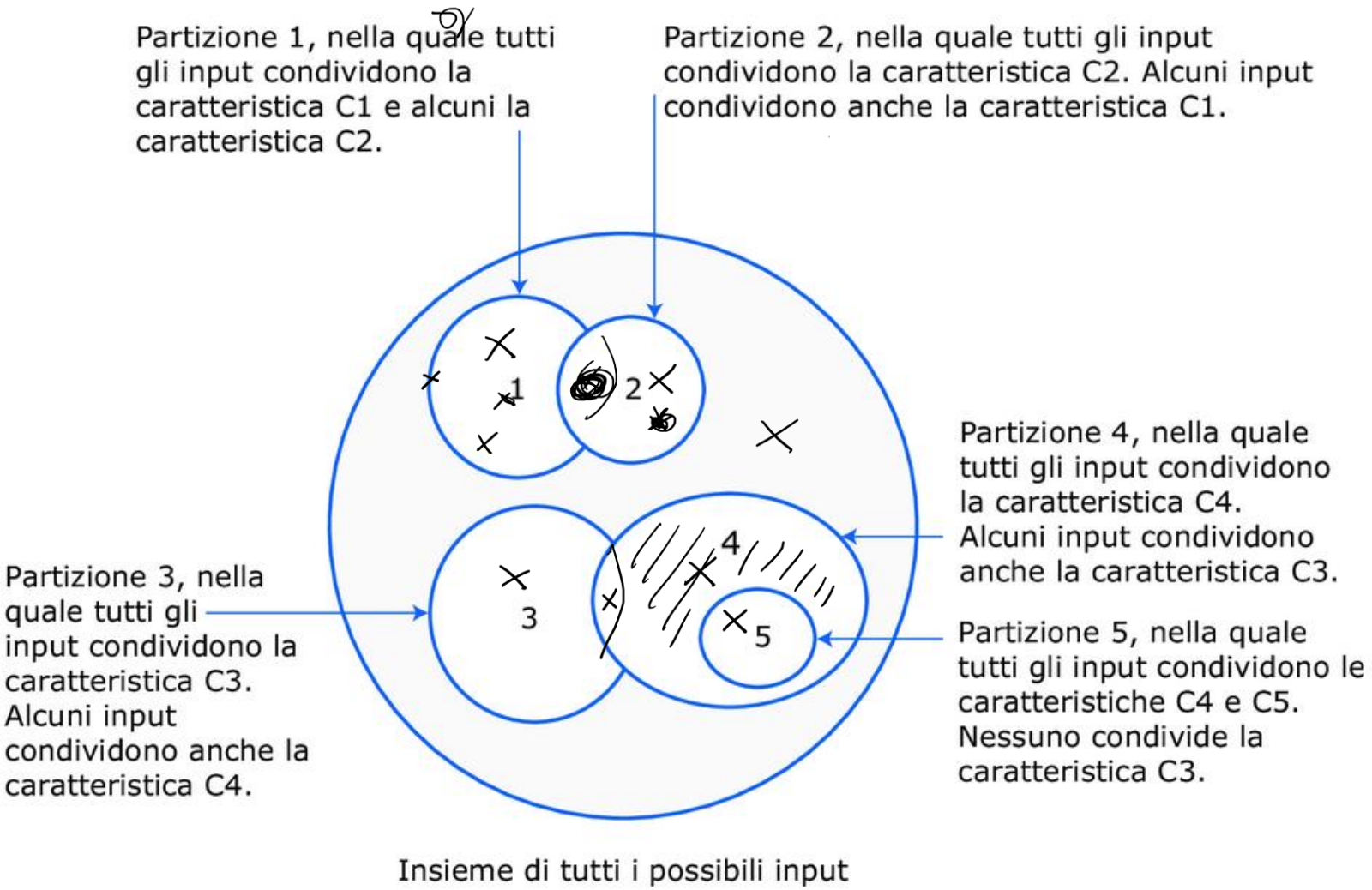
Equivalence testing

- **Ipotesi del test di equivalenza**: *i sistemi di solito si comportano in modo simile per tutti i membri di una classe*.
- Per testare il comportamento di una classe di equivalenza, è sufficiente testarne solo un membro della classe.

Il test di equivalenza consiste in due fasi:

1. identificazione le classi di equivalenza
   - I possibili input sono partizionati in classi di equivalenza
     - **partizioni di equivalenza** = set di input che verranno trattati allo stesso modo nel codice.
     - Le partizioni di equivalenza identificate non dovrebbero includere solo quelle contenenti input che producono i valori corretti. Occorre anche identificare le "**partizioni di non correttezza**" in cui gli input sono deliberatamente errati.
2. viene selezionato un test case per ogni classe

Equivalence testing



Le partizioni aiutano ad individuare le classi di equivalenza

Equivalence testing

I seguenti criteri sono utilizzati per determinare le classi di equivalenza:

Copertura. Ogni possibile input appartiene a una delle classi di equivalenza.

Disarticolazione. Nessun input appartiene a più di una classe di equivalenza.

Rappresentazione. Se si osserva un malfunzionamento quando un particolare membro di una classe di equivalenza viene utilizzato come input, lo stesso malfunzionamento può essere osservato utilizzando come input qualsiasi altro membro della classe.

Esempio di equivalence testing

```
calcola_prezzo(eta):
    if eta < 0 or eta > 120:
        return "Errore"
    elif eta < 12:
        return "Biglietto bambino"
    elif eta < 65:
        return "Biglietto adulto"
    else:
        return "Biglietto anziano"
```

L'input è `eta` (età di una persona).
Il dominio valido: 0 ≤ eta ≤ 120.

	Classe	Valore scelto	Output atteso
Non valida	`eta < 0`	`-5`	"Errore"
Valida	`0 ≤ eta < 12`	`5`	"Biglietto bambino"
Valida	`12 ≤ eta < 65`	`30`	"Biglietto adulto"
Valida	`65 ≤ eta ≤ 120`	`80`	"Biglietto anziano"
Non valida	`eta > 120`	`150`	"Errore"

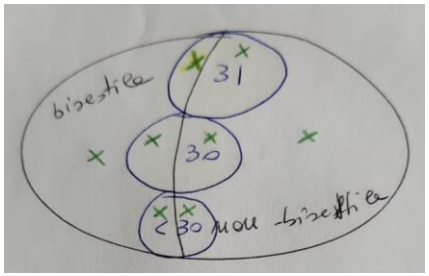
Esempio di equivalence testing

```
class MyGregorianCalendar {
...
public static int getNumDaysInMonth(int month,int year) {…}
...
}
```

Equivalence class	Value for month input	Value for year input
Months with 31 days, non–leap years	7 (July)	1901
Months with 31 days, leap years	7 (July)	1904
Months with 30 days, non–leap years	6 (June)	1901
Month with 30 days, leap year	6 (June)	1904
Month with 28 or 29 days, non–leap year	2 (February)	1901
Month with 28 or 29 days, leap year	2 (February)	1904

3 classi di eq x mese: 31,30 e <30 gg

2 classi di eq x anno: bisestili o no



+ Input non validi: int neg per mese e anno, int >12 per mese

Boundary testing

Piuttosto che selezionare qualsiasi elemento nella classe di equivalenza, il **boundary testing** richiede che gli elementi siano selezionati al *bordo* della classe di equivalenza (valori limite della classe)

- Valori aggiunti dal boundary testing
  - Febbraio valore limite per i falsi bisestili
  - Valori limite nella rappresentazione del mese

Equivalence class	Value for month input	Value for year input
Leap years divisible by 400	2 (February)	2000
Non–leap years divisible by 100	2 (February)	1900
Nonpositive invalid months	0	1291
Positive invalid months	13	1315

Un altro esempio di equivalence testing

```
def controllonome (s):
        # controlla che un nome contenga solo caratteri alfabetici,
        # i nomi devono essere lunghi da 2 a 40 caratteri
        # non sono consentite stringhe tra apici e --
        namex = r”^[a-zA-Z][a-zA-Z-’]{1,39}$”
        if re.match (namex, s):
                if re.search (“’.*’”, s) or re.search (“--”, s):
                        return False
                 else:
                        return True
        else:
                return False
```

Partizione di equivalenza	Caratteristica
Nomi corretti 1	Gli input includono solo caratteri alfabetici e sono lunghi da 2 a 40 caratteri.
Nomi corretti 2	Gli input includono solo caratteri alfabetici, trattini o apostrofi e sono lunghi da 2 a 40 caratteri.
Nomi non corretti 1	Gli input sono lunghi da 2 a 40 caratteri ma contengono caratteri non consentiti.
Nomi non corretti 2	Gli input includono solo caratteri consentiti, ma sono lunghi solo 1 carattere o più di 40.
Nomi non corretti 3	Gli input sono lunghi da 2 a 40 caratteri, ma il primo carattere è un trattino o un apostrofo.
Nomi non corretti 4	Gli input contengono solo caratteri validi e sono lunghi da 2 a 40 caratteri, ma includono doppi trattini, testo tra apici o entrambi.

Altre linee guida di James Whittaker

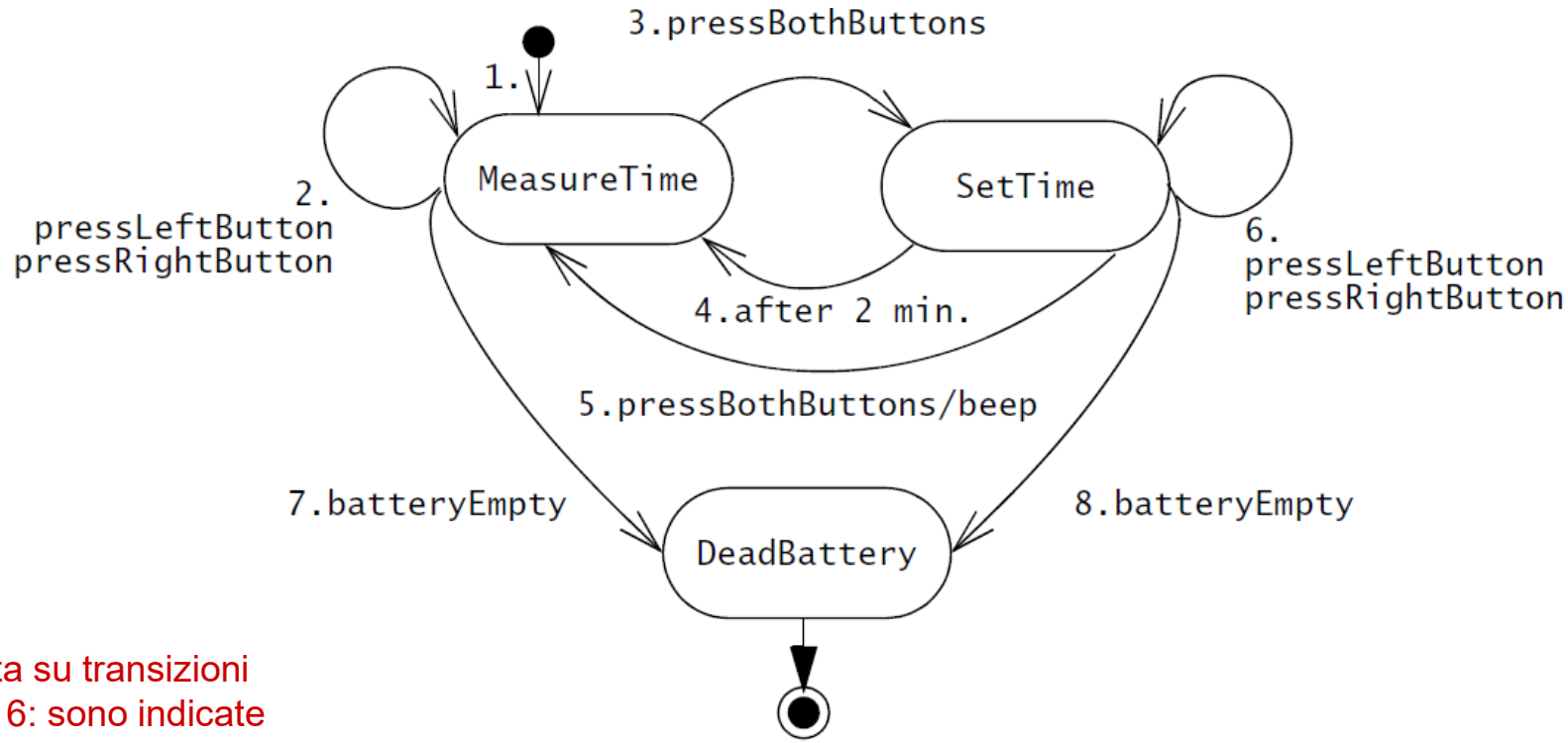
Linea guida	Spiegazione
Testare casi limite (di frontiera)	Se la partizione ha dei limiti minimi e massimi (come la lunghezza di stringhe, numeri e così via), si scelgano input ai limiti dell'intervallo.
Forzare errori	Si scelgano input che forzino il sistema a generare tutti i messaggi d'errore. Si scelgano input che dovrebbero generare output non validi.
Riempire i buffer	Si scelgano input che causino l'overflow di tutti i buffer di input.
Ripetersi	Si ripeta più volte lo stesso input o la stessa serie di input.
Overflow e underflow	Se il programma esegue calcoli numerici, si scelgano input che causano il calcolo di numeri molto grandi o molto piccoli.
Non dimenticare null e zero	Se il programma usa puntatori o stringhe, si testi sempre con puntatori e stringhe null. Se si usano sequenze, si testi con una sequenza vuota. Per gli input numerici, si testi sempre con zero.
Tenere il conto	In presenza di liste e trasformazioni di liste, si tenga conto del numero di elementi in ogni lista e si verifichi che siano coerenti dopo ogni trasformazione.
Uno è diverso	Se il programma opera con sequenze, si testi sempre con sequenze che hanno un solo valore.

State-based testing

Nel contesto di una classe, lo **state-based testing** (o state transition testing) deriva i casi di test da macchina a stati (UML) per la classe.

- **Test case**: sequenza di *stimoli o eventi* per testare:
  - *la copertura di path di esecuzione (ossia di sequenze di transizioni)*
  - *la raggiungibilità di stati specifici*
    - *la raggiungibilità di tutti gli stati finali*
- **Test set :** insieme di casi di test adeguati per raggiungere la copertura richiesta

Macchina a stati di un orologio



Nota su transizioni 2 e 6: sono indicate insieme 2 transizioni diverse

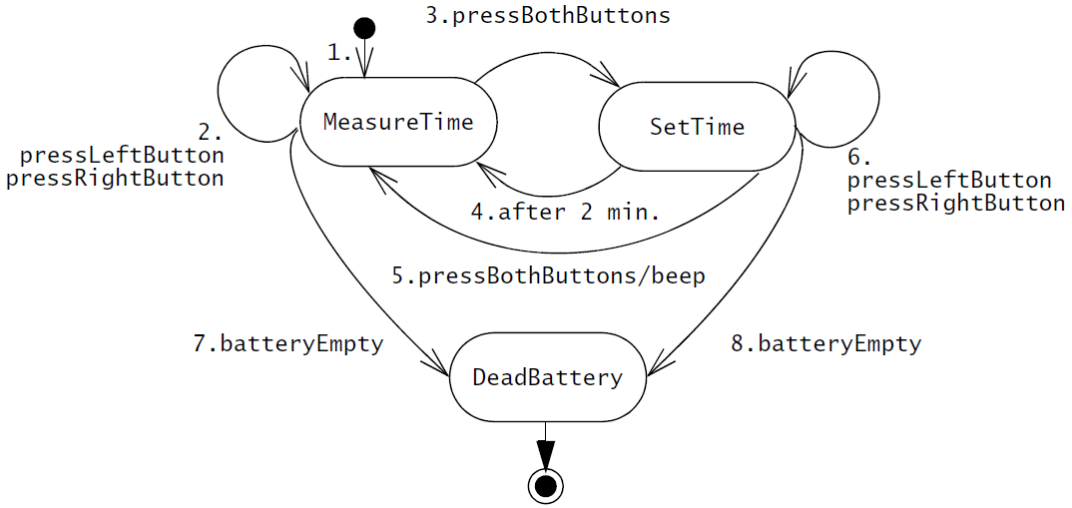
Stati e transizioni

#	Stato sorgente	Evento / Azione	Stato destinazione
1	(iniziale) →	–	MeasureTime
2	MeasureTime	pressLeftButton / pressRightButton	MeasureTime
3	MeasureTime	pressBothButtons	SetTime
4	SetTime	after 2 min.	MeasureTime
5	MeasureTime	pressBothButtons / beep	MeasureTime
6	SetTime	pressLeftButton / pressRightButton	SetTime
7	MeasureTime	batteryEmpty	DeadBattery
8	SetTime	batteryEmpty	DeadBattery

Singoli test case e copertura offerta

Test Case	Sequenza di input/eventi	Cammino attraversato	Copre le transizioni
TC1	Avvio sistema	Start → MeasureTime	1
TC2	pressLeftButton	MeasureTime → MeasureTime	2
TC3	pressBothButtons	MeasureTime → SetTime	3
TC4	after 2 min.	SetTime → MeasureTime	4
TC5	pressBothButtons	MeasureTime → MeasureTime (con beep)	5
TC6	pressRightButton (mentre in SetTime)	SetTime → SetTime	6
TC7	batteryEmpty (mentre in MeasureTime)	MeasureTime → DeadBattery	7
TC8	batteryEmpty (mentre in SetTime)	SetTime → DeadBattery	8

Test set di copertura



Raggiungibilità di DeadBattery
Test case = <Start, pressBothButtons, batteryEmpty>
Path coperto: Init→ MeasureTime → SetTime → DeadBattery
Transizioni coperte: 1,3,8
Test set = {<Start, batteryEmpty>} , <Start, pressBothButtons, batteryEmpty>, <Start, pressLeftButtons, pressBothButtons, batteryEmpty>}

Copertura di Transizione 5
Test set = {<Start, pressBothButtons, pressBothButtons>} (minimale)
Path coperto: Init→ MeasureTime → SetTime → MeasureTime

Copertura di Path: MeasureTime → SetTime → MeasureTime → MeasureTime
Test set = {<pressBothButtons, after 2 min, pressLeftButtons>}
Transizioni coperte: 3, 4, 2

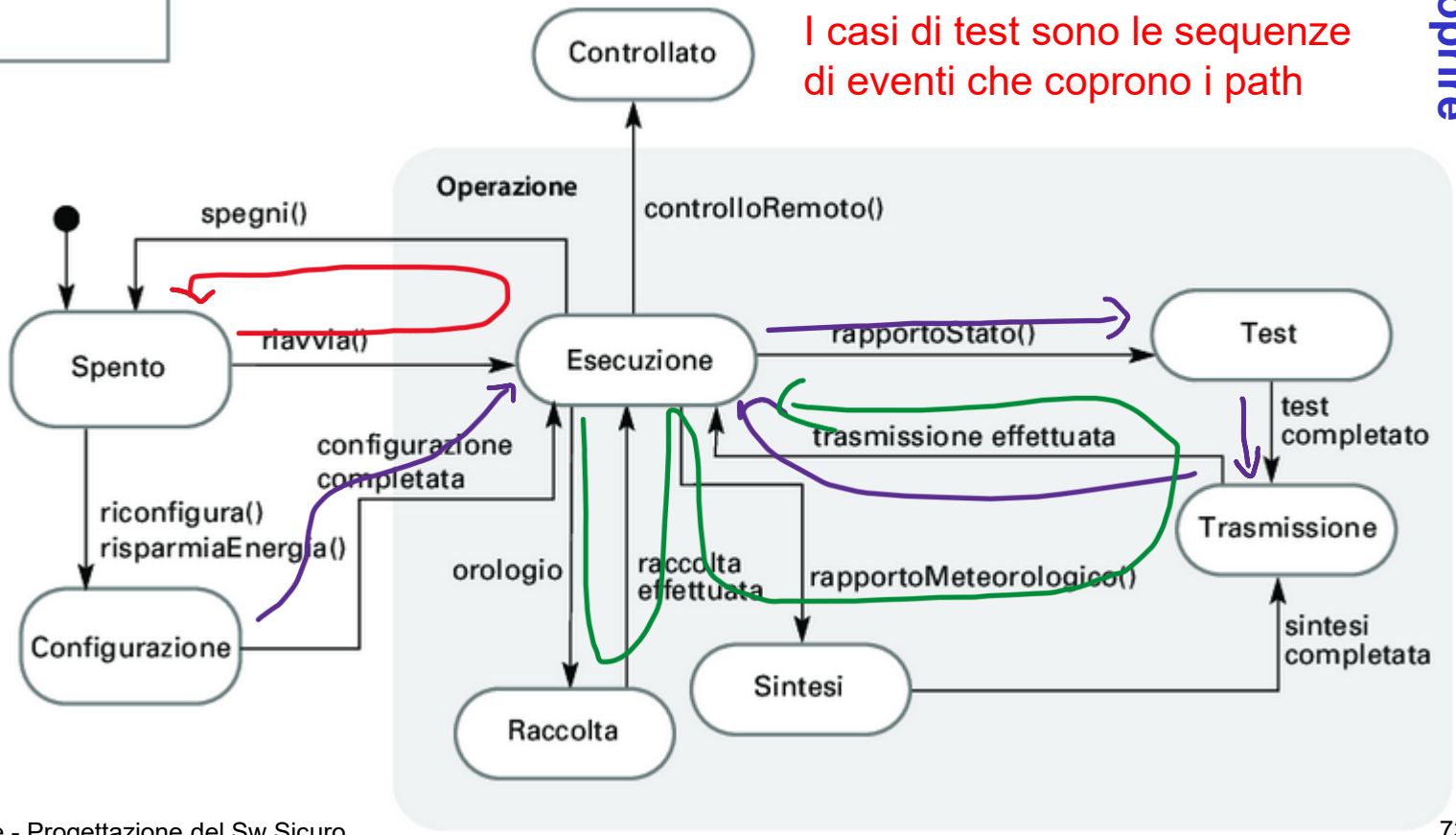
Esempio di state-based testing

StazioneMeteorologica
identificatore
rapportoMeteorologico() rapportoStato() risparmiaEnergia() controlloRemoto() riconfigura() riavvia() spegni()

Spento → Esecuzione → Spento

Configurazione → Esecuzione → Test → Trasmissione → Esecuzione

Esecuzione → Raccolta → Esecuzione → Sintesi → Trasmissione → Esecuzione

Path da coprire

I casi di test sono le sequenze di eventi che coprono i path